



NAO Move

Documentation Technique Complète

Application Python 3 de contrôle du robot NAO V6
Pilotage, navigation autonome par BFS, visualisation temps réel

Version 1.0 — Stage Commandement Itinéraire

Table des matières

1.	Présentation générale	3
2.	Architecture de l'application	4
3.	Fichier principal — main.py	5
4.	Le Bridge — nao_bridge.py	7
5.	Module Éditeur — editor_module.py	10
6.	Module Vue NAO — view_module.py	14
7.	Module Logs — logs_module.py	17
8.	Gestionnaire d'onglets — tab_manager.py	19
9.	Navigation — scene_idc.py	20
10.	Serveur NAO — serveur_nao.py	24
11.	Compilation en .exe	27
12.	Idées d'améliorations	28

1. Présentation générale

NAO Move est une application de bureau développée en Python 3 avec la bibliothèque tkinter. Elle permet de concevoir graphiquement des cartes de navigation pour le robot NAO V6, de calculer automatiquement un itinéraire optimal entre deux points, de piloter le robot (réel ou simulé dans Chorégraphe), et de visualiser ses déplacements en temps réel.

L'application a été développée dans le cadre d'un projet universitaire (L3 Informatique) visant à automatiser la navigation d'un robot NAO dans un environnement défini par une grille 2D.

1.1 Fonctionnalités principales

- Éditeur de scène graphique avec zoom, scrolling, undo/redo, drag & drop
- Calcul automatique de chemin par algorithme BFS (Breadth-First Search)
- Pilotage du robot NAO via socket (local ou réseau)
- Visualisation temps réel du déplacement du robot sur la carte
- Système de logs filtrable et exportable
- Compilation possible en .exe autonome via PyInstaller

1.2 Technologies utilisées

Technologie	Rôle	Version
Python 3.11+	Langage principal, interface, navigation	3.11 / 3.14
tkinter / ttk	Interface graphique native	Inclus avec Python
Python 2.7 (Chorégraphe)	Contrôle du robot via naoqi	2.7.18
naoqi (ALProxy)	API robot NAO — mouvements, posture	2.8
socket	Communication inter-process	Stdlib
threading / queue	Concurrence, queues thread-safe	Stdlib
subprocess	Lancement de scripts en arrière-plan	Stdlib
json	Sérialisation des scènes	Stdlib
PyInstaller	Compilation en .exe autonome	6.x

1.3 Contrainte Python 2 / Python 3

La contrainte majeure du projet est la coexistence de deux versions de Python. La bibliothèque naoqi, nécessaire pour contrôler le robot NAO, n'est disponible que pour Python 2.7 sur Windows, via le Python embarqué dans Chorégraphe. L'interface et la logique de navigation tournent en Python 3. La communication entre les deux se fait via socket TCP local (port 9561).

Architecture deux-process

Process 1 (Python 3.11) : interface graphique + calcul de chemin BFS + bridge

Process 2 (Python 2.7 Chorégraphe) : serveur socket + commandes robot via naoqi

Communication : socket TCP localhost port 9561

2. Architecture de l'application

2.1 Structure des fichiers

```
NAO_Move/
■■■ main.py                # Point d'entrée – fenêtre principale + tutoriel
■■■ scene_idc.py           # Client navigation (Python 3)
■■■ serveur_nao.py        # Serveur robot (Python 2 + naoqi)
■■■ scene-icon.ico         # Icône application
■■■ build.bat              # Script compilation .exe
■■■ NAO_Move.spec          # Configuration PyInstaller
■■■ modules/
    ■■■ __init__.py
    ■■■ base_module.py     # Classe abstraite des modules
    ■■■ tab_manager.py     # Gestion onglets ttk.Notebook
    ■■■ nao_bridge.py      # Bus de communication inter-modules
    ■■■ editor_module.py   # Onglet éditeur de scène
    ■■■ view_module.py     # Onglet visualisation robot
    ■■■ logs_module.py     # Onglet logs
```

2.2 Flux de données

Le flux de données de l'application suit un schéma en étoile centré sur le NaoBridge. Tous les modules communiquent exclusivement via le bridge, jamais directement entre eux. Cela garantit un couplage faible et facilite l'ajout de nouveaux modules.

Émetteur	Canal	Destinataire(s)
EditorModule	bridge.set_scene() — scene_queue	ViewModule (_poll_scene)
EditorModule	bridge.launch_nao() — subprocess	scene_idc.py
scene_idc.py (stdout)	[NAO_POS] x y — position_queue	ViewModule (_on_position_update)
scene_idc.py (stdout)	[NAO_CHEMIN] [...] — chemin_callbacks	ViewModule (_on_chemin_update)
Tous modules	bridge.log() — log_queue	LogsModule (_recevoir_log)
bridge	update_status()	Barre de statut (NaoApp)
scene_idc.py (socket)	TCP port 9561	serveur_nao.py

2.3 Patron de conception

L'application utilise plusieurs patrons de conception classiques :

- **Observer / Callback** : le bridge maintient des listes de callbacks. Chaque module s'enregistre pour recevoir les événements qui l'intéressent (positions, logs, chemin).
- **Module / Plugin** : chaque onglet hérite de BaseModule et implémente _build(). Ajouter un nouvel onglet se fait en créant une classe, sans modifier le code existant.
- **Producer-Consumer** : les threads de subprocess produisent dans des queues thread-safe, le thread principal (tkinter) consomme via root.after() toutes les 100ms.

- **Bridge** : NaoBridge découple complètement les modules. Aucun module ne connaît les autres directement.

3. Fichier principal — main.py

main.py est le point d'entrée de l'application. Il contient deux éléments principaux : la fonction `afficher_tutoriel()` et la classe `NaoApp`.

3.1 Fonction `resource_path()`

```
def resource_path(relative_path):
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, relative_path)
    return os.path.join(os.path.dirname(__file__), relative_path)
```

Cette fonction résout les chemins de fichiers embarqués. Quand PyInstaller compile l'application en `.exe`, tous les fichiers ressources (icône, scripts) sont extraits dans un dossier temporaire référencé par `sys._MEIPASS`. Sans cette fonction, l'exé ne trouverait pas l'icône ni les scripts.

3.2 Fonction `afficher_tutoriel()`

Crée une fenêtre popup modale (`grab_set()` bloque la fenêtre principale) avec un guide de démarrage. Le contenu est défini dans la liste `etapes`, une liste de tuples (titre, contenu). Modifier le tutoriel revient à modifier cette liste.

```
etapes = [
    ("1 - Prérequis",
     "Ouvrir Chorégraphe..."),
    ("2 - Éditeur de scène – les symboles",
     "X = mur / obstacle\n■ = case praticable..."),
    # Ajouter/supprimer des tuples ici
]
```

Le centrage de la popup est calculé relativement à la fenêtre principale (`root.wininfo_x()`, `root.wininfo_y()`), et non à l'écran, pour garantir qu'elle apparaît bien au centre même si l'application est déplacée. `popup.transient(root)` la maintient au premier plan.

3.3 Classe `NaoApp`

`NaoApp` est le chef d'orchestre de l'application. Son `__init__` réalise les opérations suivantes dans l'ordre :

- Configure la fenêtre (titre, taille minimale, icône, géométrie, état agrandi via `state('zoomed')`)
- Instancie `NaoBridge` — le canal de communication partagé
- Instancie `TabManager` — le gestionnaire d'onglets
- Instancie les trois modules dans l'ordre : `EditorModule`, `ViewModule`, `LogsModule`
- Enregistre chaque module comme onglet via `tab_manager.add_tab()`
- Crée la barre de statut en bas avec le label de statut et le bouton ■ Aide
- Démarre le bridge (`bridge.start(root)` lance le polling des queues)
- Programme l'affichage du tutoriel 300ms après le démarrage

3.4 Barre de statut

La barre de statut en bas de la fenêtre affiche le message de statut courant (mis à jour par `bridge.update_status()`) et un bouton ■ Aide qui rouvre le tutoriel à tout moment. Le callback est

enregistré via `bridge.set_status_callback()`.

3.5 Fermeture propre

```
def _on_close(self):  
    self.bridge.stop()    # Arrête le subprocess scene_idc si actif  
    self.root.destroy()   # Ferme la fenêtre
```

La fermeture est interceptée par protocol('WM_DELETE_WINDOW') pour s'assurer que le subprocess de navigation est bien terminé avant de quitter.

4. Le Bridge — nao_bridge.py

nao_bridge.py est le composant central de l'architecture. Il joue le rôle de bus de messages entre tous les modules et gère le lancement du subprocess de navigation.

4.1 Fonctions utilitaires

get_resource_path(filename)

Identique à resource_path() dans main.py. Retourne le chemin correct vers un fichier embarqué, que l'on soit en mode script ou en mode .exe PyInstaller.

get_python_executable()

Retourne le chemin vers l'exécutable Python 3 à utiliser pour lancer scene_idc.py. En mode script, sys.executable suffit. En mode .exe, sys.executable pointe vers le .exe lui-même (pas un interpréteur Python), donc la fonction cherche python dans le PATH via shutil.which(), puis dans des emplacements courants.

4.2 Attributs de NaoBridge

Attribut	Type	Rôle
log_queue	queue.Queue	File de messages de log à dispatcher
position_queue	queue.Queue	File de positions NAO (x, y)
scene_queue	queue.Queue	File de mises à jour de scène
_log_callbacks	list	Fonctions abonnées aux logs
_position_callbacks	list	Fonctions abonnées aux positions
_chemin_callbacks	list	Fonctions abonnées au chemin BFS
_status_callback	function	Fonction de mise à jour barre statut
_process	subprocess.Popen	Process scene_idc.py en cours
scene_courante	list[list]	Scène JSON actuellement éditée
chemin_courant	str	Chemin fichier JSON actuel

4.3 Système de callbacks

Le bridge implémente le patron Observer. Chaque module s'enregistre pour recevoir les événements qui l'intéressent :

```
# Enregistrement (dans les modules)
self.bridge.add_log_callback(self._recevoir_log)
self.bridge.add_position_callback(self._on_position_update)
self.bridge.add_chemin_callback(self._on_chemin_update)

# Émission (depuis n'importe où)
self.bridge.log("Message", "INFO")
```

```
self.position_queue.put((x, y))
```

4.4 Le polling — méthode `_poll()`

tkinter n'est pas thread-safe : on ne peut pas modifier des widgets depuis un thread secondaire. Le bridge résout ce problème avec un polling toutes les 100ms dans le thread principal via `root.after()` :

```
def _poll(self):
    if not self._running:
        return
    # Vider la queue de logs et dispatcher aux callbacks
    try:
        while True:
            level, msg = self.log_queue.get_nowait()
            for cb in self._log_callbacks:
                cb(level, msg)
    except queue.Empty:
        pass
    # Même chose pour les positions
    try:
        while True:
            pos = self.position_queue.get_nowait()
            for cb in self._position_callbacks:
                cb(pos)
    except queue.Empty:
        pass
    self._root.after(100, self._poll) # Rappel dans 100ms
```

Les threads secondaires (subprocess reader) déposent dans les queues. Le thread principal vide les queues et appelle les callbacks toutes les 100ms. C'est le seul endroit où l'on touche aux widgets depuis les données des threads.

4.5 Lancement de la navigation — `launch_nao()`

```
def launch_nao(self, scene_path):
    script = get_resource_path("scene_idc.py")
    python_exe = get_python_executable()
    env = os.environ.copy()
    env["NAO_SCENE_PATH"] = scene_path

    creation_flags = subprocess.CREATE_NO_WINDOW if sys.platform == "win32" else 0

    self._process = subprocess.Popen(
        [python_exe, script],
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True, env=env,
        cwd=os.path.dirname(script),
        creationflags=creation_flags # Cache le terminal cmd
    )
```

Points clés de `launch_nao()` :

- `CREATE_NO_WINDOW` cache le terminal cmd Windows — sans ça, une fenêtre noire s'ouvre à chaque lancement
- `NAO_SCENE_PATH` est passé en variable d'environnement pour que `scene_idc.py` sache quel fichier charger
- `stdout=PIPE + stderr=STDOUT` capturent toute la sortie du subprocess dans un seul flux
- Un thread daemon lit ce flux ligne par ligne et parse les messages spéciaux

Parsing du stdout

Préfixe	Action
[NAO_POS] x y	Extrait les coordonnées et les met dans <code>position_queue</code>
[NAO_CHEMIN] [...]	Parse le JSON du chemin BFS et appelle les <code>chemin_callbacks</code>
Autre	Détermine le niveau (ERROR si 'error' dans le texte, INFO sinon) et met dans <code>log_queue</code>

5. Module Éditeur — editor_module.py

L'éditeur de scène est le module le plus complexe. Il permet de dessiner une carte 2D, de la sauvegarder/charger en JSON, et de lancer la navigation. Il hérite de BaseModule.

5.1 Constantes et configuration

```
SYMBLES = ["X", "■", "N", "E", "■"]
COULEURS = {
    "X": "black",    # Mur
    "■": "yellow",   # Case praticable
    "N": "blue",     # Position NAO
    "E": "orange",   # Orientation initiale
    "■": "green",    # Objectif
    "■": "white"     # Case vide
}
LARGEUR_MIN, LARGEUR_MAX = 5, 30
HAUTEUR_MIN, HAUTEUR_MAX = 5, 30
```

5.2 Layout de l'éditeur

L'éditeur utilise un layout grid tkinter à 3 colonnes et 4 lignes :

Zone	Position grid	Contenu
Barre zoom	row=0, col=0..2	Slider zoom + label
Panneau gauche	row=1, col=0	Dimensions + champs + bouton Appliquer
Canvas scrollable	row=1, col=1 + row=2, col=1	Grille de dessin + scrollbars
Panneau droite	row=1, col=2	Boutons symboles + undo/redo
Barre bas	row=3, col=0..2	Enregistrer / Charger / Clear / Lancer NAO

5.3 Système de grille (Canvas + Frame)

La grille n'est pas dessinée sur un Canvas tkinter.Canvas, mais construite avec des widgets tk.Label dans un tk.Frame posé sur un Canvas. Cette approche permet d'utiliser les bindings classiques (,) sur chaque cellule individuellement.

```
# Structure canvas/frame
self.canvas = tk.Canvas(self.frame, ...)
self.canvas_frame = tk.Frame(self.canvas, ...) # frame dans le canvas
self.canvas_window = self.canvas.create_window((0,0),
    window=self.canvas_frame, anchor="nw")

# Chaque cellule est un Label
lbl = tk.Label(self.canvas_frame, bg="white", borderwidth=1, relief="solid")
lbl.grid(row=y, column=x, sticky="nsew")
```

Centrage automatique

Quand la grille est plus petite que le canvas (zoom faible ou petite grille), `centrer_grille()` calcule les offsets et repositionne le `canvas_window` :

```
def centrer_grille(self):
    cw, ch = self.canvas.wininfo_width(), self.canvas.wininfo_height()
    fw, fh = self.canvas_frame.wininfo_reqwidth(), self.canvas_frame.wininfo_reqheight()
    x = max(0, (cw - fw) // 2)
    y = max(0, (ch - fh) // 2)
    self.canvas.coords(self.canvas_window, x, y)
    self.canvas.configure(scrollregion=self.canvas.bbox("all"))
```

5.4 Zoom

Le zoom est implémenté via `minsize` sur les colonnes/lignes du grid. Chaque unité de zoom correspond à une taille de case en pixels :

```
def _appliquer_zoom(self, taille):
    font_size = max(8, taille // 3)
    for x in range(self.largeur):
        self.canvas_frame.grid_columnconfigure(x, weight=0, minsize=taille)
    for y in range(self.hauteur):
        self.canvas_frame.grid_rowconfigure(y, weight=0, minsize=taille)
    for y in range(self.hauteur):
        for x in range(self.largeur):
            self.labels[y][x].config(width=1, height=1, font=("Arial", font_size))
```

5.5 Dessin — cliquer() et _glisser()

Un clic appelle `_debut_action()` qui sauvegarde l'état (undo) puis `cliquer()`. Le drag appelle directement `cliquer()` sans sauvegarde, pour qu'un seul undo efface tout le tracé du drag.

```
def cliquer(self, x, y):
    if self.mode == "effacer":
        self.scene[y][x] = "■"
        self.labels[y][x].config(bg="white", text="")
    else:
        sym = SYMBOLES[self.selection]
        # Unicité de N, E, ■ : effacer l'ancienne occurrence
        if sym in ("E", "N", "■"):
            for iy in range(self.hauteur):
                for ix in range(self.largeur):
                    if self.scene[iy][ix] == sym:
                        self.scene[iy][ix] = "■"
                        self.labels[iy][ix].config(bg="white", text="")
        self.scene[y][x] = sym
        self.labels[y][x].config(bg=COULEURS[sym], ...)
        self.bridge.set_scene(self.scene, self.chemin_courant)
```

L'unicité de N, E et ■ est garantie : si on en place un nouveau, l'ancien est effacé automatiquement. C'est critique car `scene_idc.py` utilise `trouver_position()` qui retourne seulement la première occurrence.

5.6 Undo / Redo

```
def _sauvegarder_etat(self):
    self.historique.append([ligne[:] for ligne in self.scene])
```

```

        self.futur.clear()

def undo(self):
    if not self.historique: return
    self.futur.append([ligne[:] for ligne in self.scene])
    self.scene = self.historique.pop()
    self.refresh()

```

La scène est une liste de listes. La copie `[ligne[:] for ligne in self.scene]` crée une copie profonde (shallow copy de chaque ligne). Sans ça, toutes les entrées de l'historique pointeraient vers le même objet.

5.7 Lancement NAO — `lancer_nao()`

```

def _lancer_nao(self):
    if self.chemin_courant:
        # Sauvegarde TOUJOURS la scène courante avant de lancer
        with open(self.chemin_courant, "w", encoding="utf-8") as f:
            json.dump(self.scene, f, ensure_ascii=False, indent=2)
        self.bridge.set_scene(self.scene, self.chemin_courant)
        self.bridge.launch_nao(self.chemin_courant)
    else:
        self.bridge.log("Sauvegardez d'abord...", "WARN")

```

La sauvegarde systématique avant le lancement est critique : `scene_idc.py` lit le fichier JSON depuis le disque. Sans cette sauvegarde, NAO suivrait l'ancienne version du fichier et non ce qui est affiché à l'écran.

5.8 Chargement adaptatif

Quand on charge un fichier JSON dont les dimensions diffèrent de la grille actuelle, la grille est automatiquement recrée aux nouvelles dimensions, et le zoom est recalculé pour que la grille entière soit visible dans le canvas.

6. Module Vue NAO — view_module.py

Le module Vue NAO affiche en temps réel la scène et la position du robot. Il utilise un vrai Canvas tkinter pour dessiner, contrairement à l'éditeur qui utilise des Labels.

6.1 Dictionnaire de couleurs

Symbole	Couleur	Signification
X	#222222 (noir)	Mur / obstacle
■ (hors chemin)	#aeaeae (gris)	Case praticable non utilisée
■ (sur chemin BFS)	#ffe066 (jaune)	Case faisant partie de l'itinéraire
■	#33aa44 (vert)	Objectif
N	#3366cc (bleu)	Position départ NAO
E	#ff8800 (orange)	Orientation initiale
■ / vide	#f0f0f0 (gris clair)	Case vide
♦ (robot)	#ff3333 (rouge)	Position actuelle du robot

6.2 Polling de la scène — poll_scene()

```
def _poll_scene(self):
    try:
        while True:
            event_type, data = self.bridge.scene_queue.get_nowait()
            if event_type == "scene_update":
                self.scene = data
                self._redessiner()
    except Exception:
        pass
    self.frame.after(300, self._poll_scene)
```

Le polling de la scène est distinct du polling principal du bridge (100ms). Il tourne toutes les 300ms car les mises à jour de scène sont moins fréquentes que les positions. Quand l'éditeur modifie la scène, il appelle `bridge.set_scene()` qui dépose dans `scene_queue`. Le ViewModule la récupère ici et redessine.

6.3 Callbacks de position et chemin

```
def _on_position_update(self, pos):
    x, y = pos
    self.nao_pos = (x, y)
    self.label_pos.config(text="Position NAO : ({} , {})".format(x, y))
    self._redessiner()

def _on_chemin_update(self, positions):
    self.chemin_positions = positions # Liste [[x,y], [x,y], ...]
    self._redessiner()
```

Ces deux méthodes sont enregistrées comme callbacks dans le bridge. Elles sont appelées dans le thread principal via le mécanisme de polling, donc il est sûr de modifier les widgets directement.

6.4 Méthode `_redessiner()`

`_redessiner()` efface et retrace entièrement le canvas à chaque mise à jour. C'est simple et suffisant pour les fréquences de mise à jour en jeu (quelques fois par seconde). Pour des besoins haute fréquence, un système de mise à jour partielle serait nécessaire.

```
chemin_set = {(p[0], p[1]) for p in self.chemin_positions}

for y in range(hauteur):
    for x in range(largeur):
        sym = self.scene[y][x]
        couleur = COULEURS.get(sym, "#f0f0f0")

        # Cases ■ hors chemin : grises
        if (x, y) not in chemin_set and sym == "■":
            couleur = "#aeaeae"
        # Cases ■ sur le chemin : jaunes
        if (x, y) in chemin_set and sym == "■":
            couleur = "#ffe066"

        self.canvas.create_rectangle(x1, y1, x2, y2, fill=couleur, ...)
```

`chemin_set` est un ensemble Python (set) pour des lookups $O(1)$ au lieu de $O(n)$ avec une liste. Sur une grille $30 \times 30 = 900$ cases, ça représente jusqu'à 900 lookups par dessin.

6.5 Dessin du robot

```
if self.nao_pos:
    nx, ny = self.nao_pos
    x1, y1 = nx * t + 4, ny * t + 4
    x2, y2 = x1 + t - 8, y1 + t - 8
    self.canvas.create_oval(x1, y1, x2, y2,
        fill="#ff3333", outline="white", width=2)
    self.canvas.create_text(nx*t + t//2, ny*t + t//2,
        text="♦", fill="white", font=("Arial", max(8, t//3), "bold"))
```

Le robot est représenté par un cercle rouge avec le symbole ♦ centré dedans. Les $+4/-8$ créent une marge de 4px par rapport aux bords de la case pour que le cercle soit légèrement plus petit que la case.

7. Module Logs — logs_module.py

Le module Logs affiche un historique coloré de tous les événements de l'application avec filtrage par niveau et export.

7.1 Zone de texte

Les logs sont affichés dans un widget tk.Text en mode disabled (lecture seule). Pour écrire dedans, on le passe temporairement en mode normal, on insère, puis on le remet disabled :

```
def _afficher_entree(self, ts, level, msg):
    self.text.config(state="normal")
    self.text.insert("end", "[{}] ".format(ts), "TIME")    # tag TIME
    self.text.insert("end", "{:<6} ".format(level), level) # tag INFO/WARN/ERROR
    self.text.insert("end", msg + "\n", "MSG")             # tag MSG
    self.text.config(state="disabled")
    if self.auto_scroll_var.get():
        self.text.see("end")
```

Chaque insertion utilise un tag de couleur différent. Les tags sont configurés une fois à la création :

```
self.text.tag_config("INFO", foreground="#6ec9f0") # Bleu clair
self.text.tag_config("WARN", foreground="#f5c842") # Jaune
self.text.tag_config("ERROR", foreground="#f07070") # Rouge
self.text.tag_config("TIME", foreground="#666666") # Gris
self.text.tag_config("MSG", foreground="#d4d4d4") # Blanc cassé
```

7.2 Filtrage

Tous les logs sont conservés dans self.tous_les_logs (liste de tuples (ts, level, msg)). Quand on change le filtre, la zone est effacée et réaffichée en ne gardant que les entrées du bon niveau. C'est simple et efficace pour les volumes de logs attendus.

7.3 Export

L'export utilise `filedialog.asksaveasfilename()` pour choisir l'emplacement, puis écrit chaque entrée au format [HH:MM:SS] LEVEL message.

8. Gestionnaire d'onglets — tab_manager.py

TabManager encapsule un widget `ttk.Notebook` et gère l'ajout d'onglets. La version actuelle est volontairement simple, sans détachement des onglets.

8.1 Code complet

```
class TabManager:
    def __init__(self, root):
        style = ttk.Style()
        style.configure("TNotebook.Tab",
            padding=[12, 6], font=("Arial", 10))
        self.notebook = ttk.Notebook(root)
        self.notebook.pack(fill="both", expand=True, padx=4, pady=4)
        self.tabs = {}

    def add_tab(self, title, module):
        self.tabs[title] = module
        self.notebook.add(module.frame, text=title)

    def get_detached_windows(self):
        return [] # Compatibility stub
```

`get_detached_windows()` retourne une liste vide pour des raisons de compatibilité avec le code de fermeture dans `NaoApp.on_close()`. Si le détachement est réimplémenté dans le futur, cette méthode retournera les fenêtres détachées.

8.2 BaseModule

```
class BaseModule:
    def __init__(self, tab_manager, bridge):
        self.tab_manager = tab_manager
        self.bridge = bridge
        self.frame = tk.Frame(tab_manager.notebook)
        self._build() # Méthode abstraite

    def _build(self):
        raise NotImplementedError
```

Chaque module hérite de `BaseModule` et implémente `_build()`. Le frame est créé dans le notebook pour être directement intégrable comme onglet. Le bridge est passé à la construction pour que chaque module puisse communiquer.

9. Navigation — scene_idc.py

scene_idc.py est le script de navigation. Il tourne en subprocess Python 3, lit la scène JSON, calcule le chemin optimal par BFS, puis pilote le robot via socket en communiquant avec serveur_nao.py.

9.1 Chargement de la scène

```
scene_path = os.environ.get("NAO_SCENE_PATH", "scene.json")
with open(scene_path, "r", encoding="utf-8") as f:
    scene = json.load(f)
```

Le chemin du fichier JSON est transmis via la variable d'environnement NAO_SCENE_PATH, définie par nao_bridge.py au moment du lancement du subprocess. Ce mécanisme évite d'avoir à passer des arguments en ligne de commande.

9.2 Émission de données vers l'interface

```
def _pos(x, y):
    print("[NAO_POS] {} {}".format(x, y))
    sys.stdout.flush()

def _emit_chemin(positions):
    print("[NAO_CHEMIN] {}".format(json.dumps(positions)))
    sys.stdout.flush()
```

La communication avec l'interface se fait via stdout. Le bridge lit ce stdout ligne par ligne et parse les préfixes. sys.stdout.flush() est impératif : sans ça, Python bufferise la sortie et l'interface ne reçoit pas les messages en temps réel.

9.3 Algorithme BFS — suivre_chemin_scene()

Le BFS (Breadth-First Search / Parcours en largeur) est l'algorithme de recherche de chemin utilisé. Il garantit de trouver le chemin le plus court en nombre de cases.

```
def suivre_chemin_scene():
    start = (NAO[0], NAO[1])
    but = tuple(trouver_position("■"))
    queue = deque([(start, [start])])
    visite = {start}

    while queue:
        (x, y), chemin_actuel = queue.popleft()
        if (x, y) == but:
            return [[p[0], p[1]] for p in chemin_actuel]
        for dx, dy in [(0,1),(0,-1),(1,0),(-1,0)]:
            nx, ny = x + dx, y + dy
            if (nx, ny) not in visite:
                if scene[ny][nx] in ("■", "■"): # Cases praticables
                    visite.add((nx, ny))
                    queue.append(((nx, ny), chemin_actuel + [(nx, ny)]))
    return [list(start)] # Aucun chemin trouvé
```

Détails d'implémentation :

- deque (double-ended queue) est utilisé à la place de list pour popleft() en O(1) au lieu de O(n)

- visite est un set pour des lookups O(1)
- Le chemin complet est stocké à chaque noeud — inefficace en mémoire mais simple. Pour de grandes grilles, on stockerait un dictionnaire parent au lieu
- Seules les cases ■ et ■ sont praticables. Les cases X, 0, N, E bloquent

9.4 Conversion chemin -> commandes

```
def deduire_commandes(positions, orientation_initiale):
    DIR = {(0,-1): 0, (1,0): 1, (0,1): 2, (-1,0): 3}
    orientation = orientation_initiale # 0=Nord 1=Est 2=Sud 3=Ouest

    for i in range(1, len(positions)):
        dx, dy = positions[i][0]-positions[i-1][0], positions[i][1]-positions[i-1][1]
        direction_cible = DIR[(dx, dy)]
        diff = (direction_cible - orientation) % 4

        if diff == 0: commandes.append("avant")
        elif diff == 1: commandes.append("droite"); orientation += 1
        elif diff == 3: commandes.append("gauche"); orientation -= 1
        elif diff == 2:
            commandes.append("rotation") # 180°
            commandes.append("avant")
```

Le système de direction encode les 4 orientations en entiers (0=Nord, 1=Est, 2=Sud, 3=Ouest). La différence modulo 4 donne la rotation nécessaire : 1 = tourner à droite, 3 = tourner à gauche (équivalent à -1 mod 4), 2 = faire demi-tour.

9.5 Gestion de l'orientation — snapshot

Un bug subtil existe dans la gestion de l'orientation : quand on exécute une commande et qu'on vérifie la prochaine, la variable globale orientation a déjà été mutée. La solution est de prendre un snapshot avant la mutation :

```
orientation_apres_commande = orientation
if commande == "rotation":
    orientation_apres_commande = (orientation + 2) % 4
elif commande in ("rotation_gauche", "gauche"):
    orientation_apres_commande = (orientation - 1) % 4
elif commande in ("rotation_droite", "droite"):
    orientation_apres_commande = (orientation + 1) % 4

dx, dy = calculer_deplacement(commande) # Mute orientation
NAO[0] += dx; NAO[1] += dy

# Utilise le snapshot, pas l'orientation mutée
verifier_prochaine_case(chemin[i+1], orientation_apres_commande)
```

9.6 Détection d'obstacles et retour

Avant chaque déplacement, scene_idc.py demande au serveur de vérifier si la case cible est libre :

```
def verifier_prochaine_case(cmd, orientation_courante):
    if cmd in ("rotation", "rotation_gauche", "rotation_droite"):
        return True # Pas de déplacement, pas d'obstacle possible
```

```
cx, cy = calculer_case_cible(cmd, orientation_courante)
envoyer("verifier:[{}{}].format(cx, cy))
feedback_recu.wait()
return not obstacle_detecte.is_set()
```

Si la case est un obstacle, retour_position_initiale() rejoue toutes les commandes en sens inverse selon le dictionnaire INVERSES :

```
INVERSES = {
    "rotation": "rotation",      # 180° → 180°
    "avant": "arriere",
    "arriere": "avant",
    "gauche": ("arriere", "rotation_droite"), # Reculer puis tourner à droite
    "droite": ("arriere", "rotation_gauche"),
}
```

10. Serveur NAO — serveur_nao.py

serveur_nao.py est le script qui s'exécute avec le Python embarqué dans Chorégraphe (Python 2.7 + naoqi). Il fait le pont entre le socket TCP et les commandes physiques du robot.

Important

Ce script doit être ouvert et lancé dans Chorégraphe AVANT de cliquer Lancer NAO dans l'application. Il ne peut pas être lancé directement depuis l'interface car il nécessite Python 2.7 + naoqi, non disponibles en Python 3.

10.1 Initialisation

```
IP = "127.0.0.1"    # IP du robot (localhost en simulation)
PORT_NAO = 9559     # Port NAOqi standard
PORT_SERVER = 9561  # Port socket entre scene_idc et serveur_nao

motion = ALProxy("ALMotion", IP, PORT_NAO)
posture = ALProxy("ALRobotPosture", IP, PORT_NAO)
life    = ALProxy("ALAutonomousLife", IP, PORT_NAO)
```

ALProxy crée un proxy vers un service NAOqi sur le robot. Les trois services utilisés sont ALMotion (déplacements), ALRobotPosture (mise en position), et ALAutonomousLife (comportements autonomes, désactivé pendant la navigation).

10.2 Réception de la scène

```
buffer_scene = ""
while "\n" not in buffer_scene:
    buffer_scene += conn.recv(4096).decode()
scene_json, reste = buffer_scene.split("\n", 1)
scene = json.loads(scene_json)
```

La scène est transmise par scene_idc.py dès la connexion, avant toute commande. Cela permet au serveur de répondre aux requêtes verifier:[x,y] en consultant la scène locale.

10.3 Dictionnaire de mouvements

```
MOUVEMENTS = {
    "avant": (lambda: motion.moveTo(0.3, 0, 0),      "NAO a avance\n"),
    "arriere": (lambda: motion.moveTo(-0.3, 0, 0),   "NAO a recule\n"),
    "gauche": (lambda: (motion.moveTo(0.001, 0, math.pi/2),
                        time.sleep(1),
                        motion.moveTo(0.3, 0, 0)),    "NAO a tourne a gauche et avance\n"),
    "droite": (lambda: (motion.moveTo(0.001, 0, -math.pi/2),
                       time.sleep(1),
                       motion.moveTo(0.3, 0, 0)),    "NAO a tourne a droite et avance\n"),
    "rotation": (lambda: motion.moveTo(0, 0, math.pi), "NAO a effectue la rotation 180\n"),
}
```

Chaque commande est un tuple (lambda_action, message_feedback). Les commandes gauche et droite combinent rotation ET avance dans la même commande car c'est le comportement naturel en navigation : tourner puis avancer d'une case.

motion.moveTo(x, y, theta) déplace le robot de x mètres en avant, y mètres sur le côté, et theta radians de rotation. 0.3m correspond environ à une case de grille réelle.

10.4 Boucle de traitement

```
while not stopper:
    if not select.select([conn], [], [], 1.0)[0]:
        continue # Timeout 1s – évite le blocage

    data = conn.recv(1024).decode()
    buffer += data
    lignes = buffer.split("\n")
    buffer = lignes[-1] # Garde le fragment incomplet

    for commande in lignes[:-1]:
        if commande.startswith("verifier:"):
            # Vérification locale de la scène
            resultat = verifier_case(cx, cy)
            conn.send("{}\n".format(resultat).encode())
        elif commande in MOUVEMENTS:
            action, feedback = MOUVEMENTS[commande]
            action() # Exécute le mouvement
            conn.send(feedback.encode()) # Confirme l'exécution
```

select.select() évite le blocage total en attendant des données. Le timeout de 1s permet de vérifier régulièrement si stopper a été mis à True. Le système de buffer avec split('\n') gère correctement les messages fragmentés par TCP.

11. Compilation en .exe

L'application peut être compilée en un seul fichier .exe autonome avec PyInstaller. Une fois compilé, l'utilisateur n'a pas besoin d'installer Python.

11.1 Fichier NAO_Move.spec

Le fichier .spec est la recette de compilation PyInstaller. Il liste explicitement les fichiers à embarquer :

```
datas=[
    ('scene-icon.ico', '.'),      # Icône
    ('scene_idc.py', '.'),       # Script navigation (lancé en subprocess)
    ('serveur_nao.py', '.'),     # Script robot (pour référence)
    ('modules', 'modules'),      # Dossier complet des modules
]
```

11.2 Pourquoi embarquer scene_idc.py

Même si scene_idc.py est lancé en subprocess, il doit être accessible sur le disque car subprocess.Popen() exécute un fichier physique. PyInstaller extrait les datas dans sys._MEIPASS au lancement, et get_resource_path() sait chercher là.

11.3 Commandes

```
# Installation PyInstaller
pip install pyinstaller

# Compilation via le .spec (recommandé)
pyinstaller NAO_Move.spec

# Ou via le script automatique
build.bat
```

Le .exe final se trouve dans dist/NAO Move.exe. Une fois compilé, tout le dossier source peut être supprimé — seul le .exe est nécessaire pour l'utilisation. Python 3 doit néanmoins rester installé sur la machine car scene_idc.py en a besoin.

12. Idées d'améliorations

12.1 Fonctionnelles

- **Multiples scénarios** : gestion de plusieurs fichiers de scène ouverts simultanément dans des onglets
- **Éditeur de symboles** : permettre à l'utilisateur de créer ses propres types de cases avec couleurs et comportements personnalisés
- **Replay** : rejouer un itinéraire enregistré depuis les logs
- **Simulation hors robot** : mode simulation sans Chorégraphe, avec un robot virtuel qui suit le chemin en temps réel
- **Pathfinding alternatif** : implémenter A* en plus du BFS pour comparer les chemins selon différents critères (distance euclidienne vs nombre de tournants)
- **Obstacles dynamiques** : permettre de modifier la scène pendant la navigation et recalculer le chemin en temps réel

12.2 Techniques

- **Tests unitaires** : ajouter des tests pour le BFS, la conversion chemin→commandes, et la gestion de l'orientation
- **Configuration persistante** : sauvegarder les préférences (zoom, chemin Python Chorégraphe, dernière scène ouverte) dans un fichier de configuration
- **Historique de navigation** : sauvegarder les itinéraires parcourus avec timestamps dans un fichier pour analyse ultérieure
- **Export image** : exporter la vue NAO (carte + itinéraire) en PNG
- **Protocole de communication robuste** : remplacer le socket nu par un protocole avec accusé de réception et détection de perte de connexion

12.3 Interface

- **Thème sombre** : implémenter un thème sombre complet pour l'interface
- **Raccourcis clavier** : Ctrl+S sauvegarder, Ctrl+O ouvrir, F5 lancer NAO, etc.
- **Minimap** : afficher une vue miniature de la scène entière dans un coin de la Vue NAO pour les grandes grilles
- **Statistiques de navigation** : afficher distance parcourue, nombre de tournants, temps estimé

Note sur la compatibilité Python

La contrainte Python 2.7 / naoqi disparaîtra si SoftBank publie un SDK Python 3 complet pour Windows (actuellement disponible seulement sur Linux/Mac via le SDK qi). Si cela arrive, les deux process pourraient fusionner en un seul, éliminant la complexité du bridge socket.